

Week 3 Assignment: SQL & Excel for Data Analytics

Name: Gajanan Harinarayan Rajee

College: MGM's College of Computer Science & IT, Nanded (TYBCA, SRTMU University)

Internship: InternNova - Data Analytics Intern

Date: 30 September 2026

Tools used: SQLite (sqliteonline.com), Microsoft Excel, Google Sheets

Sample Database (used for Tasks 1-5)

```
CREATE TABLE Departments (DeptID INT PRIMARY KEY, DeptName TEXT);  
INSERT INTO Departments VALUES (1, 'IT'), (2, 'HR'), (3, 'Sales'), (4, 'Finance');
```

```
CREATE TABLE Employees (EmpID INT PRIMARY KEY, Name TEXT, DeptID INT, Salary INT);  
INSERT INTO Employees VALUES  
(1, 'Amit', 1, 50000), (2, 'Neha', 1, 60000), (3, 'Ravi', 2, 40000),  
(4, 'Sneha', 3, 55000), (5, 'Om', NULL, 35000);
```

Explanation: A database stores data in tables made of rows and columns. Two related tables, Employees and Departments, are linked by DeptID.

Task 1: Introduction to Databases & SELECT

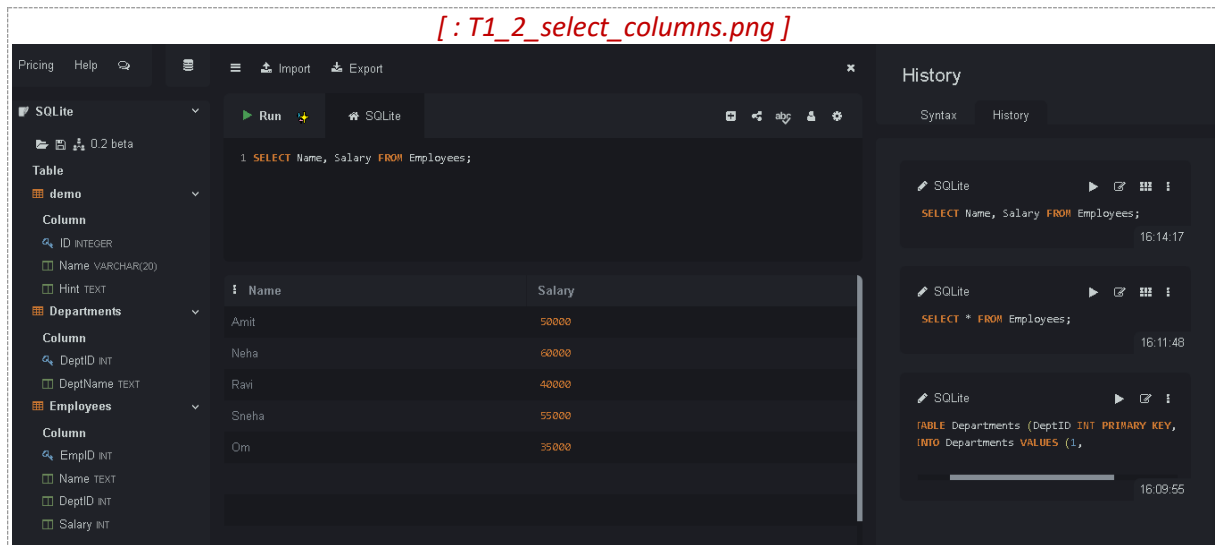
```
SELECT * FROM Employees;
```

[: T1_1_select_all.png]

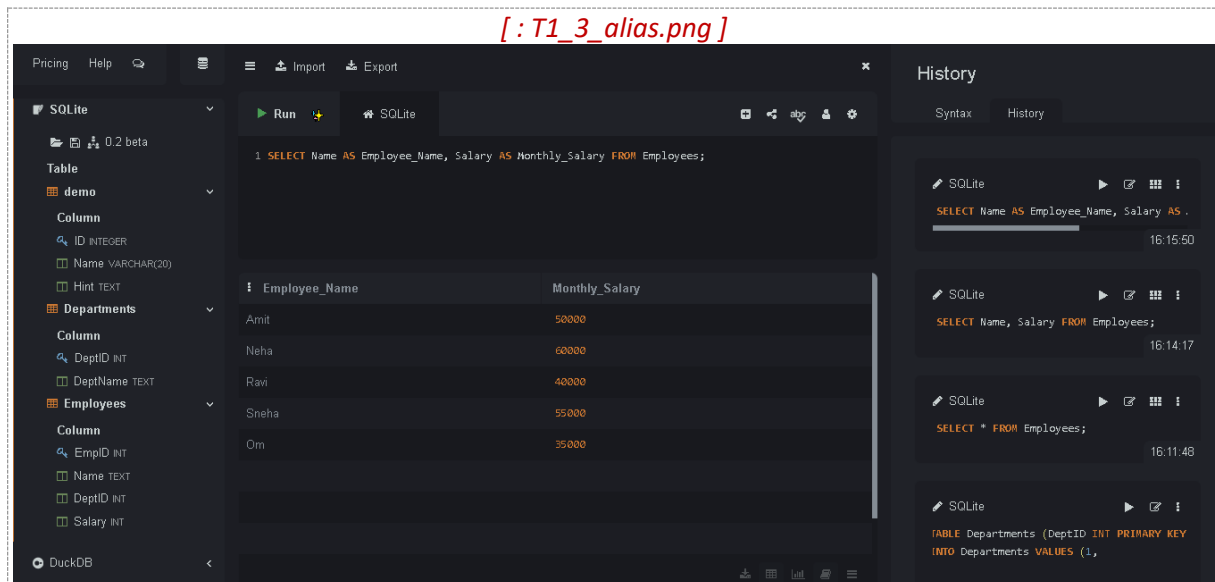
The screenshot shows the SQLiteonline.com web interface. The sidebar on the left displays the database structure for 'demo', including tables 'Departments' and 'Employees' with their respective columns and data types. The central area shows the SQL query '1 SELECT * FROM Employees;' and its results in a table with 5 rows and 4 columns: EmpID, Name, DeptID, and Salary. The results table shows data for employees Amit, Neha, Ravi, Sneha, and Om. The right sidebar shows the query history with the same query and its execution time (16:11:48).

EmpID	Name	DeptID	Salary
1	Amit	1	50000
2	Neha	1	60000
3	Ravi	2	40000
4	Sneha	3	55000
5	Om	NULL	35000

```
SELECT Name, Salary FROM Employees;
```



```
SELECT Name AS Employee_Name, Salary AS Monthly_Salary FROM Employees;
```



*Explanation: SELECT * shows all records, SELECT with column names shows only chosen columns, and AS gives a column a readable alias.*

Task 2: WHERE, ORDER BY & Aggregate Functions

```
SELECT * FROM Employees WHERE Salary > 45000;
```

[: T2_1_where_salary.png]

The screenshot shows the SQLite IDE interface. On the left, a sidebar lists the database schema: 'demo' table with columns 'ID' (INTEGER), 'Name' (VARCHAR(20)), and 'Hint' (TEXT); 'Departments' table with columns 'DeptID' (INT) and 'DeptName' (TEXT); and 'Employees' table with columns 'EmpID' (INT), 'Name' (TEXT), 'DeptID' (INT), and 'Salary' (INT). The main editor displays the SQL query: `1 SELECT * FROM Employees WHERE Salary > 45000;`. Below the query, the results are shown in a table with columns 'EmpID', 'Name', 'DeptID', and 'Salary'. The results table contains three rows: (1, Amit, 1, 50000), (2, Neha, 1, 60000), and (4, Sneha, 3, 55000). On the right, a 'History' panel shows a list of previously executed queries and their timestamps.

EmpID	Name	DeptID	Salary
1	Amit	1	50000
2	Neha	1	60000
4	Sneha	3	55000

SELECT Name, Salary FROM Employees WHERE DeptID = 1;

[: T2_2_where_dept.png]

The screenshot shows the SQLite IDE interface. The main editor displays the SQL query: `1 SELECT Name, Salary FROM Employees WHERE DeptID = 1;`. Below the query, the results are shown in a table with columns 'Name' and 'Salary'. The results table contains two rows: (Amit, 50000) and (Neha, 60000). The 'History' panel on the right shows a list of previously executed queries and their timestamps.

Name	Salary
Amit	50000
Neha	60000

SELECT * FROM Employees ORDER BY Salary DESC;

[: T2_3_order_by.png]

The screenshot shows the SQLite IDE interface. On the left, a sidebar lists the database structure: SQLite (0.2 beta) with tables 'demo', 'Departments', and 'Employees'. The 'Employees' table is expanded, showing columns: EmpID (INT), Name (TEXT), DeptID (INT), and Salary (INT). The main editor displays the SQL query: `1 SELECT * FROM Employees ORDER BY Salary DESC;`. Below the query, the results are shown in a table:

EmpID	Name	DeptID	Salary
2	Neha	1	60000
4	Sneha	3	55000
1	Amit	1	50000
3	Ravi	2	40000
5	Om	NULL	35000

```
SELECT COUNT(*) AS Total_Employees FROM Employees;
```

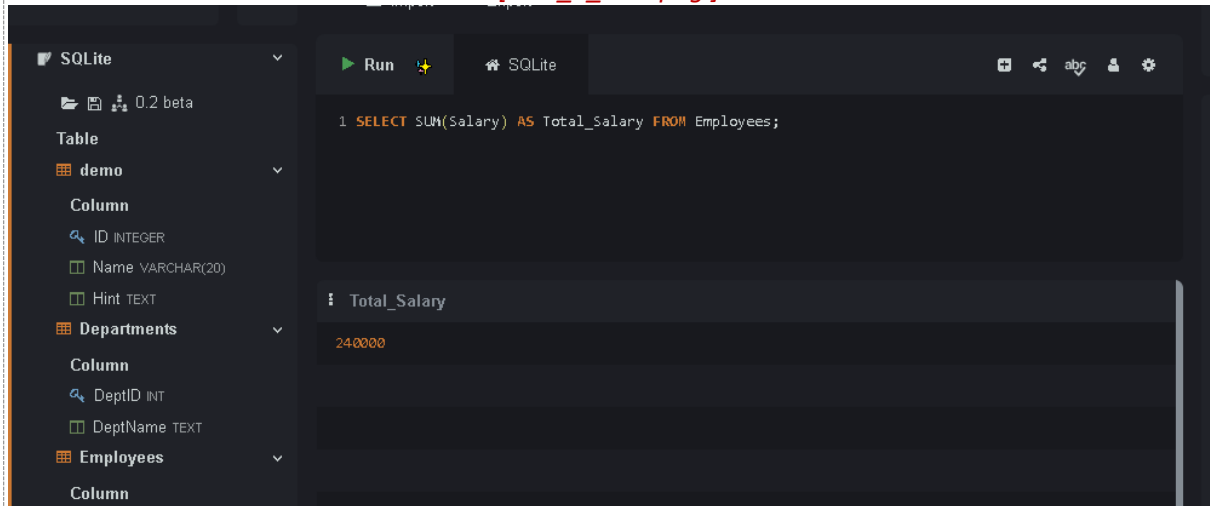
[: T2_4_count.png]

The screenshot shows the SQLite IDE interface. The main editor displays the SQL query: `1 SELECT COUNT(*) AS Total_Employees FROM Employees;`. Below the query, the results are shown in a table:

Total_Employees
5

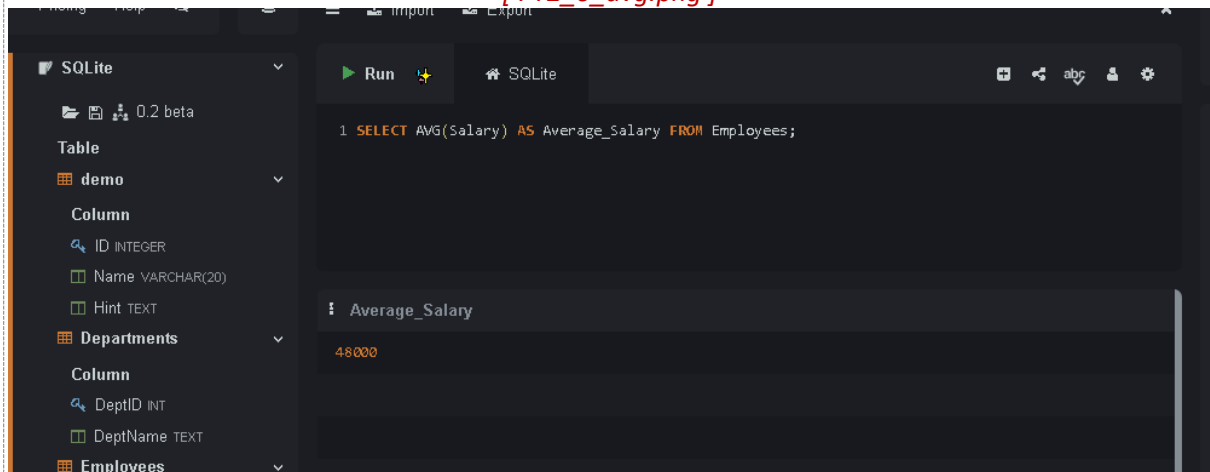
```
SELECT SUM(Salary) AS Total_Salary FROM Employees;
```

[: T2_5_sum.png]



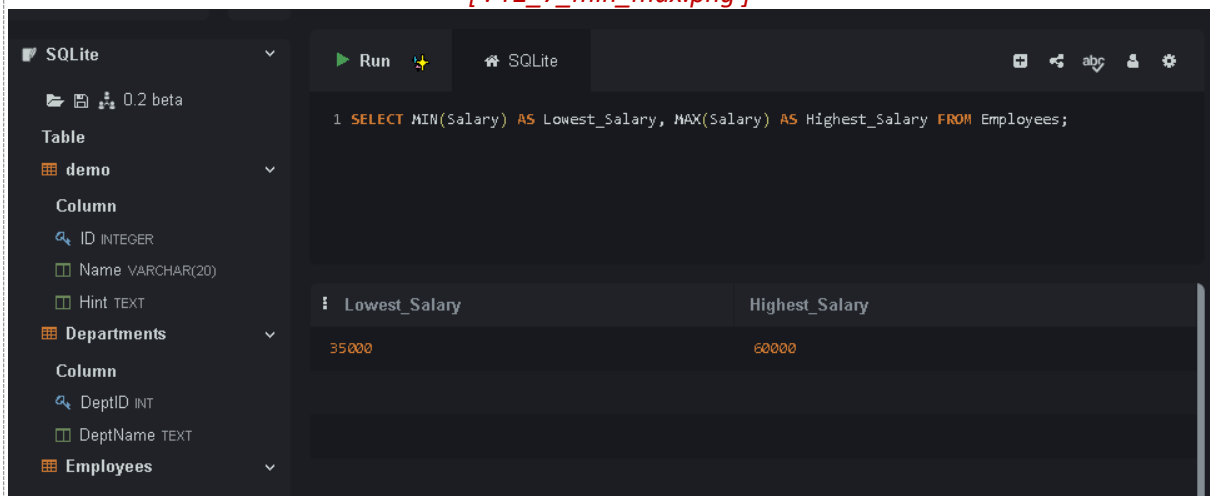
```
SELECT AVG(Salary) AS Average_Salary FROM Employees;
```

[: T2_6_avg.png]



```
SELECT MIN(Salary) AS Lowest_Salary, MAX(Salary) AS Highest_Salary FROM Employees;
```

[: T2_7_min_max.png]

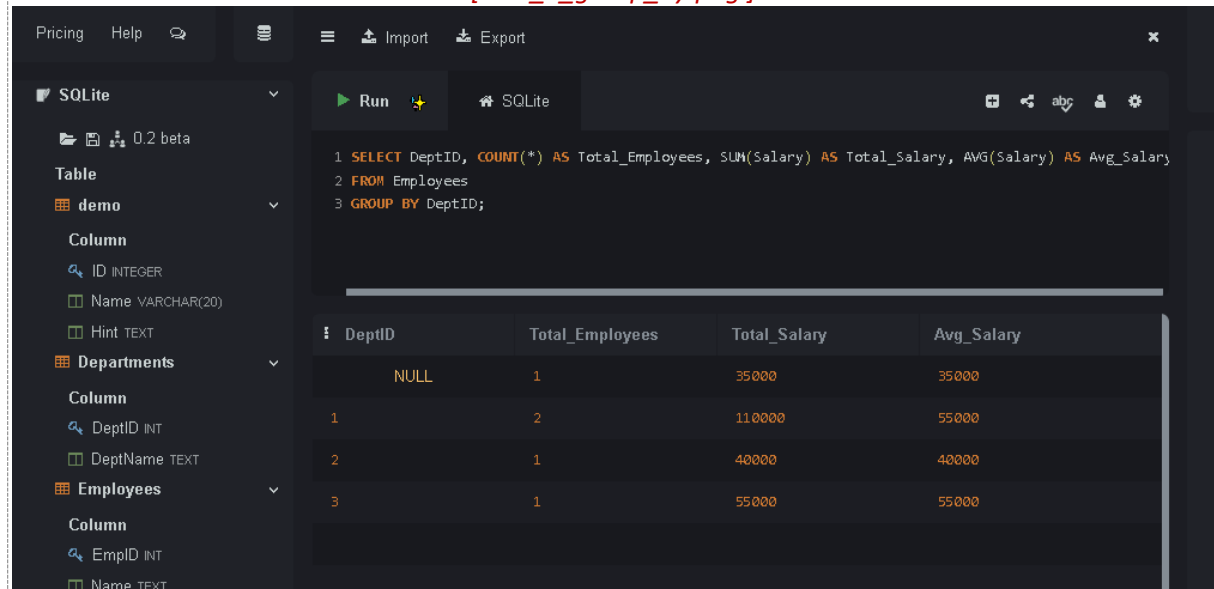


Explanation: WHERE filters rows using comparison operators (>, =), ORDER BY sorts the result, and COUNT, SUM, AVG, MIN, MAX calculate values over the Salary column.

Task 3: GROUP BY & HAVING

```
SELECT DeptID, COUNT(*) AS Total_Employees, SUM(Salary) AS Total_Salary,  
AVG(Salary) AS Avg_Salary  
FROM Employees  
GROUP BY DeptID;
```

[: T3_1_group_by.png]

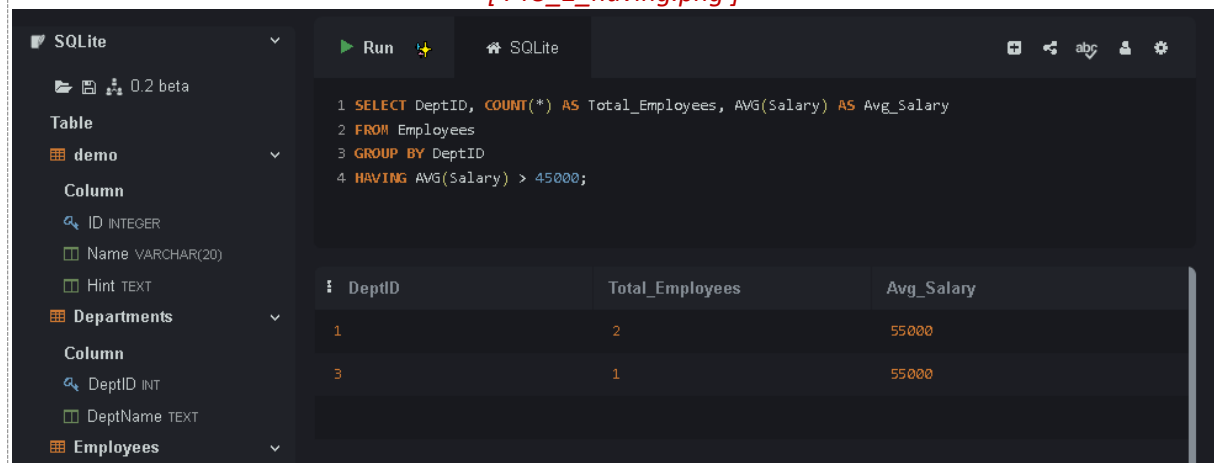


The screenshot shows the SQLite IDE interface. On the left, a sidebar lists the database schema: 'demo' table with columns 'ID INTEGER', 'Name VARCHAR(20)', 'Hint TEXT', 'Departments' (a table), and 'Employees' (a table). The 'Employees' table has columns 'EmpID INT' and 'Name TEXT'. The main area displays a SQL query:
1 SELECT DeptID, COUNT(*) AS Total_Employees, SUM(Salary) AS Total_Salary, AVG(Salary) AS Avg_Salary
2 FROM Employees
3 GROUP BY DeptID;
Below the query, the results are shown in a table with 4 columns: DeptID, Total_Employees, Total_Salary, and Avg_Salary. The data rows are: (NULL, 1, 35000, 35000), (1, 2, 110000, 55000), (2, 1, 40000, 40000), and (3, 1, 55000, 55000).

DeptID	Total_Employees	Total_Salary	Avg_Salary
NULL	1	35000	35000
1	2	110000	55000
2	1	40000	40000
3	1	55000	55000

```
SELECT DeptID, COUNT(*) AS Total_Employees, AVG(Salary) AS Avg_Salary  
FROM Employees  
GROUP BY DeptID  
HAVING AVG(Salary) > 45000;
```

[: T3_2_having.png]



The screenshot shows the SQLite IDE interface. The main area displays a SQL query:
1 SELECT DeptID, COUNT(*) AS Total_Employees, AVG(Salary) AS Avg_Salary
2 FROM Employees
3 GROUP BY DeptID
4 HAVING AVG(Salary) > 45000;
Below the query, the results are shown in a table with 3 columns: DeptID, Total_Employees, and Avg_Salary. The data rows are: (1, 2, 55000) and (3, 1, 55000).

DeptID	Total_Employees	Avg_Salary
1	2	55000
3	1	55000

Explanation: GROUP BY groups employees by DeptID and calculates count, sum and average for each group. HAVING filters the groups afterwards, so only departments with average salary above 45000 (DeptID 1 and 3) are shown.

Task 4: SQL Joins

```
SELECT e.EmpID, e.Name, d.DeptName, e.Salary  
FROM Employees e  
INNER JOIN Departments d ON e.DeptID = d.DeptID;
```

[: T4_1_inner_join.png]

The screenshot shows the SQLite IDE interface. On the left, a sidebar lists the database structure: 'demo' database with tables 'Departments' and 'Employees'. The 'Employees' table is selected, showing columns: EmpID (INTEGER), Name (VARCHAR(20)), and Salary (TEXT). The main editor displays the following SQL query:

```
1 SELECT e.EmpID, e.Name, d.DeptName, e.Salary
2 FROM Employees e
3 INNER JOIN Departments d ON e.DeptID = d.DeptID;
```

The results are shown in a table below the query:

EmpID	Name	DeptName	Salary
1	Amit	IT	50000
2	Neha	IT	60000
3	Ravi	HR	40000
4	Sneha	Sales	55000

```
SELECT e.EmpID, e.Name, d.DeptName, e.Salary
FROM Employees e
LEFT JOIN Departments d ON e.DeptID = d.DeptID;
```

[: T4_2_left_join.png]

The screenshot shows the SQLite IDE interface. The main editor displays the following SQL query:

```
1 SELECT e.EmpID, e.Name, d.DeptName, e.Salary
2 FROM Employees e
3 LEFT JOIN Departments d ON e.DeptID = d.DeptID;
```

The results are shown in a table below the query:

EmpID	Name	DeptName	Salary
1	Amit	IT	50000
2	Neha	IT	60000
3	Ravi	HR	40000
4	Sneha	Sales	55000
5	Om	NULL	35000

```
SELECT e.EmpID, e.Name, d.DeptName, e.Salary
FROM Employees e
RIGHT JOIN Departments d ON e.DeptID = d.DeptID;
```

[: T4_3_right_join.png]

The screenshot shows the SQLite IDE interface. On the left, the database schema is visible with tables 'demo', 'Departments', and 'Employees'. The 'demo' table has columns 'ID INTEGER', 'Name VARCHAR(20)', and 'Hint TEXT'. 'Departments' has 'DeptID INT', 'DeptName TEXT', and 'Hint TEXT'. 'Employees' has 'EmpID INT', 'Name TEXT', and 'DeptID INT'. The main editor shows the following SQL query:

```
1 SELECT e.EmpID, e.Name, d.DeptName, e.Salary
2 FROM Departments d
3 LEFT JOIN Employees e ON e.DeptID = d.DeptID;
```

The results are displayed in a table with the following data:

EmpID	Name	DeptName	Salary
1	Amit	IT	50000
2	Neha	IT	60000
3	Ravi	HR	40000
4	Sneha	Sales	55000
NULL	NULL	Finance	NULL

Explanation: INNER JOIN shows only rows that match in both tables (Om is excluded). LEFT JOIN shows all employees, with NULL department for Om. RIGHT JOIN shows all departments, with NULL employee for Finance. The common column is DeptID.

Task 5: SQL Subqueries

```
SELECT Name, Salary
FROM Employees
WHERE Salary > (SELECT AVG(Salary) FROM Employees);
```

[: T5_1_subquery_avg.png]

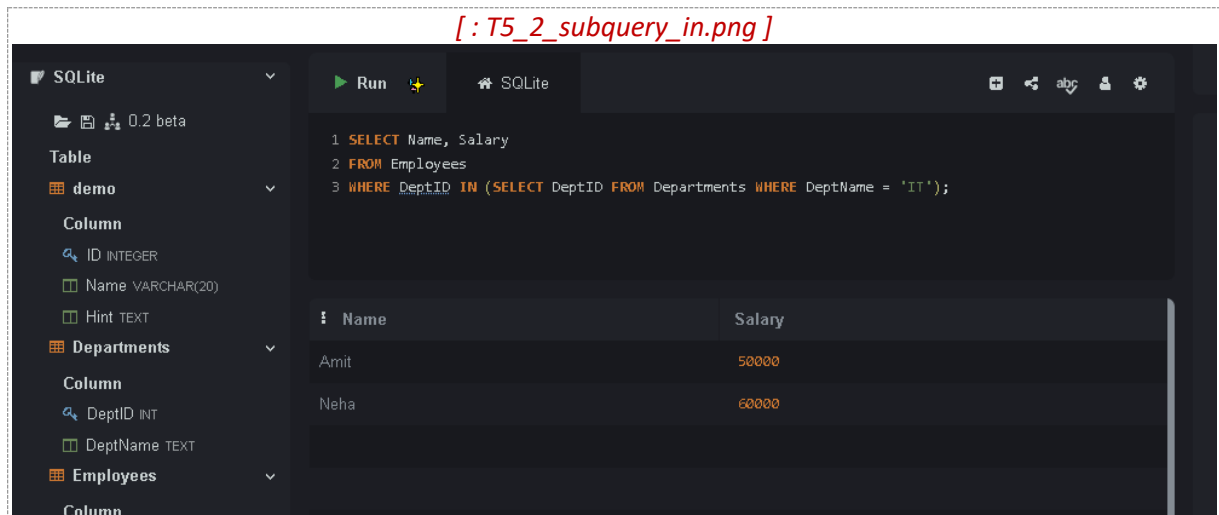
The screenshot shows the SQLite IDE interface. The main editor shows the following SQL query:

```
1 SELECT Name, Salary
2 FROM Employees
3 WHERE Salary > (SELECT AVG(Salary) FROM Employees);
```

The results are displayed in a table with the following data:

Name	Salary
Amit	50000
Neha	60000
Sneha	55000

```
SELECT Name, Salary
FROM Employees
WHERE DeptID IN (SELECT DeptID FROM Departments WHERE DeptName = 'IT');
```



Explanation: In the first query the inner SELECT finds the average salary (48000) and the outer query lists employees earning more than it. In the second, the inner query finds the IT department ID and the outer query lists its employees.

Excel Dataset (Sales_Data sheet)

Columns: Date, Region, Product, Qty, Amount (12 records, 4 regions, 5 products). A second sheet, Price_List, holds Product and Price.

Task 6: Data Formatting, Sorting & Filtering

Formatting: header row bold with colour fill, borders on all cells, currency number format on Amount, columns auto-fitted.

[: T6_1_formatted.png]

Function Library						
GET GENUINE OFFICE Your license isn't genuine, and you may be a victim of software c						
H10						
	A	B	C	D	E	F
1	Date	Region	Product	Qty	Amount	
2	9/1/2026	North	Laptop	2	₹ 100,000.00	
3	9/2/2026	South	Mouse	10	₹ 5,000.00	
4	9/3/2026	East	Keyboard	5	₹ 7,500.00	
5	9/4/2026	West	Monitor	3	₹ 36,000.00	
6	9/5/2026	North	Printer	2	₹ 16,000.00	
7	9/6/2026	South	Laptop	1	₹ 50,000.00	
8	9/7/2026	East	Monitor	2	₹ 24,000.00	
9	9/8/2026	West	Mouse	20	₹ 10,000.00	
10	9/9/2026	North	Keyboard	8	₹ 12,000.00	
11	9/10/2026	South	Printer	3	₹ 24,000.00	
12	9/11/2026	East	Laptop	1	₹ 50,000.00	
13	9/12/2026	West	Keyboard	4	₹ 6,000.00	
14						
15						
16						

Sorting: Amount sorted from largest to smallest.

I13						
	A	B	C	D	E	F
1	Date	Region	Product	Qty	Amount	
2	9/1/2026	North	Laptop	2	₹ 100,000.00	
3	9/6/2026	South	Laptop	1	₹ 50,000.00	
4	9/11/2026	East	Laptop	1	₹ 50,000.00	
5	9/4/2026	West	Monitor	3	₹ 36,000.00	
6	9/7/2026	East	Monitor	2	₹ 24,000.00	
7	9/10/2026	South	Printer	3	₹ 24,000.00	
8	9/5/2026	North	Printer	2	₹ 16,000.00	
9	9/9/2026	North	Keyboard	8	₹ 12,000.00	
10	9/8/2026	West	Mouse	20	₹ 10,000.00	
11	9/3/2026	East	Keyboard	5	₹ 7,500.00	
12	9/12/2026	West	Keyboard	4	₹ 6,000.00	
13	9/2/2026	South	Mouse	10	₹ 5,000.00	
14						
15						

[: T6_2_sorted.png]

Filtering: filter applied on Region = North.

[: T6_3_filtered.png]

A3					9/6/2026		
	A	B	C	D	E	F	G
1	Date	Regi	Produc	Qty	Amount		
2	9/1/2026	North	Laptop	2	₹ 100,000.00		
8	9/5/2026	North	Printer	2	₹ 16,000.00		
9	9/9/2026	North	Keyboard	8	₹ 12,000.00		
14							
15							
16							
17							
18							
19							

Explanation: Formatting makes data readable, sorting orders records by Amount, and filters show only the records that match a condition.

Task 7: Conditional Formatting & Excel Functions

Conditional Formatting: Amount greater than 20000 highlighted in light red.

[: T7_1_cond_format.png]

	A	B	C	D	E	F	G
1	Date	Regi	Produc	Qty	Amount		
2	9/1/2026	North	Laptop	2	₹ 100,000.00		
3	9/6/2026	South	Laptop	1	₹ 50,000.00		
4	9/11/2026	East	Laptop	1	₹ 50,000.00		
5	9/4/2026	West	Monitor	3	₹ 36,000.00		
6	9/7/2026	East	Monitor	2	₹ 24,000.00		
7	9/10/2026	South	Printer	3	₹ 24,000.00		
8	9/5/2026	North	Printer	2	₹ 16,000.00		
9	9/9/2026	North	Keyboard	8	₹ 12,000.00		
10	9/8/2026	West	Mouse	20	₹ 10,000.00		
11	9/3/2026	East	Keyboard	5	₹ 7,500.00		
12	9/12/2026	West	Keyboard	4	₹ 6,000.00		
13	9/2/2026	South	Mouse	10	₹ 5,000.00		
14							
15							
16							

IF: =IF(E2>20000,"High","Low")

[: T7_2_if.png]

	A	B	C	D	E	F	G	H	I
1	Date	Regi	Produc	Qty	Amount	category			
2	9/1/2026	North	Laptop	2	₹ 100,000.00	High			
3	9/6/2026	South	Laptop	1	₹ 50,000.00	High			
4	9/11/2026	East	Laptop	1	₹ 50,000.00	High			
5	9/4/2026	West	Monitor	3	₹ 36,000.00	High			
6	9/7/2026	East	Monitor	2	₹ 24,000.00	High			
7	9/10/2026	South	Printer	3	₹ 24,000.00	High			
8	9/5/2026	North	Printer	2	₹ 16,000.00	Low			
9	9/9/2026	North	Keyboard	8	₹ 12,000.00	Low			
10	9/8/2026	West	Mouse	20	₹ 10,000.00	Low			
11	9/3/2026	East	Keyboard	5	₹ 7,500.00	Low			
12	9/12/2026	West	Keyboard	4	₹ 6,000.00	Low			
13	9/2/2026	South	Mouse	10	₹ 5,000.00	Low			
14									
15									
16									
17									

COUNTIF: =COUNTIF(B2:B13,"North")

[: T7_3_countif.png]

	A	B	C	D	E	F	G	H	I
1	Date	Regi	Produc	Qty	Amount	category		Count of North	Total
2	9/1/2026	North	Laptop	2	₹ 100,000.00	High		3	
3	9/6/2026	South	Laptop	1	₹ 50,000.00	High		2	
4	9/11/2026	East	Laptop	1	₹ 50,000.00	High		2	
5	9/4/2026	West	Monitor	3	₹ 36,000.00	High		2	
6	9/7/2026	East	Monitor	2	₹ 24,000.00	High		2	
7	9/10/2026	South	Printer	3	₹ 24,000.00	High		2	
8	9/5/2026	North	Printer	2	₹ 16,000.00	Low		2	
9	9/9/2026	North	Keyboard	8	₹ 12,000.00	Low		1	
10	9/8/2026	West	Mouse	20	₹ 10,000.00	Low		0	
11	9/3/2026	East	Keyboard	5	₹ 7,500.00	Low		0	
12	9/12/2026	West	Keyboard	4	₹ 6,000.00	Low		0	
13	9/2/2026	South	Mouse	10	₹ 5,000.00	Low		0	
14									
15									
16									
17									

SUMIF: =SUMIF(B2:B13,"North",E2:E13)

[illegible]

Explanation: Conditional formatting highlights large sales. IF labels each sale High or Low, COUNTIF counts North records, and SUMIF adds the Amount of North records only.

Task 8: VLOOKUP & XLOOKUP

VLOOKUP: =VLOOKUP(C2,Price_List!\$A\$2:\$B\$6,2,FALSE)

[illegible]

XLOOKUP: =XLOOKUP(C2,Price List!\$A\$2:\$A\$6,Price List!\$B\$2:\$B\$6,"Not Found")

[illegible]

Note: XLOOKUP was demonstrated in Google Sheets because the installed Excel version does not support it.

Explanation: Both functions fetch the Price of each Product from Price_List. VLOOKUP needs a column number (2) and searches only left to right. XLOOKUP takes separate lookup and return ranges, needs no column number, and supports a default "Not Found" value, so it is more flexible.

Task 9: Pivot Table & Charts

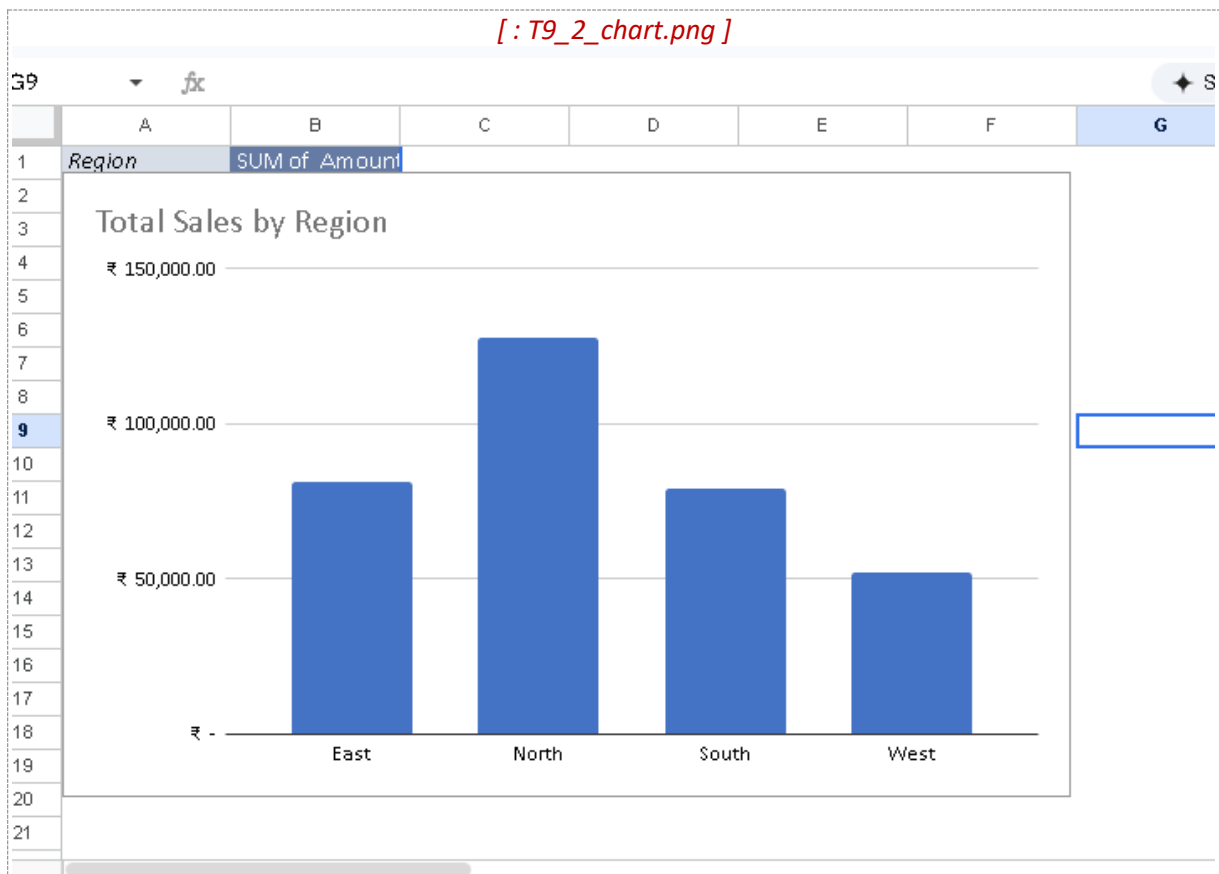
Pivot Table: Region in Rows, Sum of Amount in Values.

E4 fx

	A	B	C
1	Region	SUM of Amount	
2	East	₹ 81,500.00	
3	North	₹ 128,000.00	
4	South	₹ 79,000.00	
5	West	₹ 52,000.00	
6	Grand Total	₹ 340,500.00	
7			
8			
9			
10			
11			

[: T9_1_pivot.png]

Chart: column chart "Total Sales by Region".



Explanation: The pivot table totals Amount by Region. North has the highest sales and West the lowest, and the column chart makes this comparison easy to see.

Submission Files

Week3_SQL.sql, week3_excel.xlsx and this PDF report are available in the shared Google Drive folder.